

Einstieg in Java – Teil 1

Java-Grundlagen, Codestruktur und `main`

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

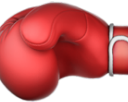
Prof. Dr. Martin Burger { 20. Oktober 2025 }

main



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Einstieg in Java

 Plan für diese Woche

 Java von Kopf bis Fuß

 Wie Java funktioniert

 Codestruktur

 main()-Methode

 Anweisungen

 Schleifen

 Verzweigungen

 Übungen

 Spaß



Einstieg in Java



Verbindungen herstellen



Tauschen Sie sich mit einer Nachbar:in aus!



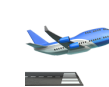
Welche aktiven Lerntechniken empfiehlt das Buch für effektives Lernen von Java?



Wie wird Java-Code geschrieben, kompiliert und ausgeführt? Welche Rolle spielt dabei die Java Virtual Machine?



Wie ist ein Java-Programm grundsätzlich aufgebaut?



Wofür verwenden Sie in Ihrem Java-Programm die Methode `main`?



Welche Fragen haben Sie zu diesem Kapitel?

1 Tauchen Sie ein: eine Kostprobe

Die Oberfläche durchbrechen



Los, das Wasser ist klasse! Wir tauchen einfach ein und schreiben etwas Code, kompilieren ihn und führen ihn aus. Wir sprechen über Syntax, Schleifen und Verzweigungen und sehen uns an, was Java so cool macht. Sie werden in Sekundenschnelle programmierbereit sein.

Java bringt Sie an neue Orte. Seit seiner bescheidenen Veröffentlichung in der (schwächlichen) Version 1.02 hat Java Programmierer mit seiner freundlichen Syntax, seinen objekt-orientierten Features, der Speicherverwaltung und vor allem mit dem Versprechen auf Portabilität verführt. Die Verlockung des **Write-once, Run-anywhere** ist einfach zu groß. Es entstand eine treue Anhängerschaft, die explosionsartig wuchs, während Programmierer gegen Bugs, Einschränkungen und vor allem dagegen kämpften, dass es schneckenlangsam war. Aber das ist schon lange her. Wenn Sie gerade erst mit Java beginnen, **haben Sie Glück**. Einige von uns mussten sich zehn Kilometer durch den Schnee kämpfen (barfuß), in jede Richtung bergauf, um selbst die trivialste Applikation ans Laufen zu bringen. *Sie* dagegen können sich von dem **geschmeidigeren, schnelleren, viel, viel mächtigeren** Java von heute tragen lassen.



Java von Kopf bis Fuß

Ihre Chance

Java von Kopf bis Fuß

 Denken Sie daran, dass Sie parallel arbeiten. Vor, während und nach der „Vorlesung“.

 **Arbeiten** Sie die entsprechenden Kapitel in der Vorwoche im Selbststudium durch!

 Die ~~Vorlesung~~ Lernveranstaltung setzt dieses voraus.

 Ich behandle nicht alles und nicht im Detail.

 Hier können Sie Fragen stellen und **Antworten erhalten.**

 In der Klausur gehe ich davon aus, dass Sie keine offenen Fragen mehr haben.



Meine Top 3

Java von Kopf bis Fuß

🧘 Immer langsam. Je mehr Sie verstehen, umso weniger müssen Sie auswendig lernen. Slow Thinkers Dig Deeper

☁ Denken Sie nach, wenn Sie vom Buch etwas gefragt werden!

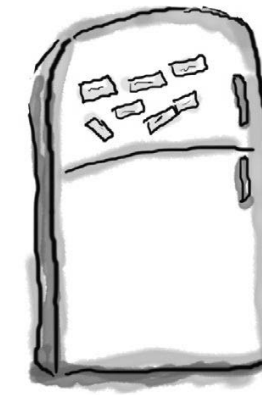
🤔 Reden Sie darüber. Laut. Test Yourself, Don't Guess Yourself

🤝 Versuchen Sie, es jemand anderem laut zu erklären!

🧑💻 Tippen Sie den Code ab und führen Sie ihn aus. Practice Makes Permanent

🤸 Spielen Sie mit den Code-Beispielen!

Einführung



Schneiden Sie dies aus und heften Sie es an Ihren Kühlschrank.

Und das können SIE tun, um sich Ihr Gehirn untertan zu machen

So, wir haben unseren Teil der Arbeit geleistet. Der Rest liegt bei Ihnen. Diese Tipps sind ein Anfang; hören Sie auf Ihr Gehirn und finden Sie heraus, was bei Ihnen funktioniert und was nicht. Probieren Sie neue Wege aus.



1 Immer langsam. Je mehr Sie verstehen, umso weniger müssen Sie auswendig lernen.

Lesen Sie nicht nur. Halten Sie inne und denken Sie nach. Wenn das Buch Sie etwas fragt, springen Sie nicht einfach zur Antwort. Stellen Sie sich vor, dass Sie das wirklich jemand *fragt*. Je gründlicher Sie Ihr Gehirn zum Nachdenken zwingen, umso größer ist die Chance, dass Sie lernen und behalten.

2 Bearbeiten Sie die Übungen. Machen Sie selbst Notizen.

Wir haben sie entworfen, aber wenn wir sie auch für Sie lösen, wäre das so, als würde jemand anderes Ihr Training für Sie absolvieren. Und sehen Sie sich die Übungen *nicht einfach nur an*. **Benutzen Sie einen Bleistift.** Es deutet vieles darauf hin, dass körperliche Aktivität *beim* Lernen den Lernerfolg erhöhen kann.

3 Lesen Sie die Abschnitte »Es gibt keine Dummten Fragen«.

Und zwar alle. Das sind keine Zusatzanmerkungen – sie gehören zum Kerninhalt! Aus den Fragen lernen Sie manchmal noch mehr als aus den Antworten.

4 Lesen Sie nicht alles am selben Ort.

Stehen Sie auf, recken Sie sich, gehen Sie herum, wechseln Sie den Stuhl, gehen Sie in einen anderen Raum. Das trägt dazu bei, dass Ihr Gehirn etwas *fühlt*, und verhindert, dass Ihr Lernerfolg an einen bestimmten Ort gebunden ist.

5 Lesen Sie dies als Letztes vor dem Schlafen. Oder lesen Sie danach zumindest nichts Anspruchsvolles mehr.

Ein Teil des Lernprozesses (vor allem die Übertragung in das Langzeitgedächtnis) findet erst statt, *nachdem* Sie das Buch zur Seite gelegt haben. Ihr Gehirn braucht Zeit für sich, um weitere Verarbeitung zu leisten. Wenn Sie in dieser Zeit etwas Neues aufnehmen, geht ein Teil dessen, was Sie gerade gelernt haben, verloren.

6 Trinken Sie Wasser. Viel.

Ihr Gehirn arbeitet am besten in einem schönen Flüssigkeitsbad. Austrocknung (zu der es schon kommen kann, bevor Sie überhaupt Durst verspüren) beeinträchtigt die kognitive Funktion.

7 Reden Sie drüber. Laut.

Sprechen aktiviert einen anderen Teil des Gehirns. Wenn Sie etwas verstehen möchten oder Ihre Chancen verbessern wollen, sich später daran zu erinnern, sagen Sie es laut. Noch besser: Versuchen Sie, es jemand anderem laut zu erklären. Sie lernen dann schneller und haben vielleicht Ideen, auf die Sie beim bloßen Lesen nie gekommen wären.

8 Hören Sie auf Ihr Gehirn.

Achten Sie darauf, Ihr Gehirn nicht zu überladen. Wenn Sie merken, dass Sie etwas nur noch überfliegen oder dass Sie das gerade erst Gelesene vergessen haben, ist es Zeit für eine Pause. Ab einem bestimmten Punkt lernen Sie nicht mehr schneller, indem Sie mehr hineinzustopfen versuchen; das kann sogar den Lernprozess stören.

9 Aber bitte mit Gefühl!

Ihr Gehirn muss wissen, dass es *um etwas Wichtiges geht*. Lassen Sie sich in die Geschichten hineinziehen. Erfinden Sie eigene Bildunterschriften für die Fotos. Über einen schlechten Scherz zu stöhnen, ist *immer noch* besser, als gar nichts zu fühlen.

10 Tippen Sie den Code ab und führen Sie ihn aus.

Tippen Sie die Codebeispiele ab und führen Sie sie aus. Dann können Sie herumexperimentieren und den Code verändern und verbessern (oder ihn kaputtmachen – das ist manchmal der beste Weg, um herauszufinden, was wirklich passiert). Bei langen Beispielen oder dem »Fertiggericht«-Code können Sie die Quellcodedateien von der Webseite zum Buch aus dem Onlinekatalog von O'Reilly herunterladen.



Sie sind hier ▶ xxv

Schreiben und Ausführen

Java von Kopf bis Fuß

</> OneCompiler

Main.java

+

Java Hello World

AI

NEW

JAVA

RUN

1 import java.util.*;

2

3 public class Main {

4 public static void main(String[] args) {

5 System.out.println("Hello, World!");

6 }

7 }

STDIN

Input for the program (Optional)

Output:

Hello, World!

Main.java — Edited

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

 Learn Download Community Contribute News Future Playground

The Java Playground

Run Clear Samples

Java 25 + Preview Features

About

1 var feature = Runtime.version().feature();

2 IO.println("🔥 Hello, Java " + feature);

3

Console >

Detailed output



Ich mute Ihnen diesen Sprung ins kalte Wasser
zu, weil ich sehr hohe Ansprüche habe...
... und weiß, dass Sie diese erfüllen können. 🎯



Wie Java funktioniert

Befehle geben

🔧 Wie Java funktioniert

- 💡 Sie möchten eine Applikation / ein Programm schreiben.
- 🤔 Wie bringen Sie den Computer dazu, sich entsprechend zu verhalten?
- 🎮 Sie „Befehlen!“ es ihm.
 - 12 34 Ihr Programm besteht aus einer Folge von Anweisungen, die festlegen, in welcher Reihenfolge der Computer was tun soll.
Imperative Programmierung



Wie Java funktioniert

Unser Ziel ist es, eine Applikation zu schreiben (in unserem Fall eine interaktive Partyeinladung), die auf jedem beliebigen von Ihren Freunden verwendeten Gerät funktionieren soll.

Dort stehen unsere Befehle drin: Anweisungen, Schleifen, Verzweigungen, ...

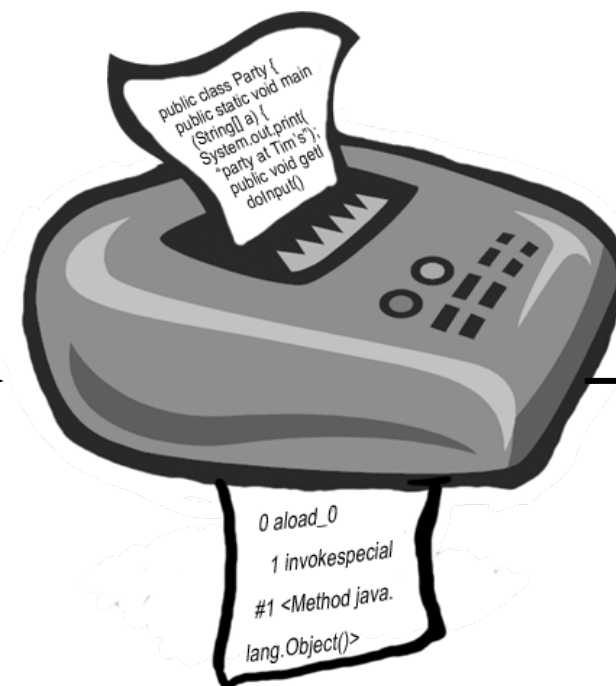


Quellcode für die interaktive Partyeinladung.

Quelle

1

Erstellen Sie ein Quelldokument. Verwenden Sie ein etabliertes Protokoll (in diesem Fall die Sprache Java).



Compiler

2

Lassen Sie Ihr Dokument durch einen Quellcode-Compiler laufen. Er überprüft den Code auf Fehler und lässt Sie erst kompilieren, wenn er sicher ist, dass alles richtig läuft.

```
Method Party()
0 aload_0
1 invokespecial #1
<Method java.lang.Object()>
4 return
```

Ausgabe (Code)

3

Der Compiler erstellt ein neues Dokument, das als Java-**Bytecode** codiert ist. Jedes Gerät, das Java ausführen kann, kann auch diese Datei interpretieren bzw. in etwas übersetzen, das es ausführen kann. Der kompilierte Bytecode ist plattformunabhängig.



Virtuelle Maschinen

4

Auf den elektronischen Spielzeugen Ihrer Freunde läuft eine als Software implementierte **virtuelle** Java-Maschine (JVM). Wenn Ihre Freunde das Programm laufen lassen, liest die virtuelle Maschine den Bytecode und führt ihn aus.

Der Computer (das Handy, ...) verhält sich wie gewünscht. 🙌

Falls keine Laufzeitfehler auftauchen... 👻

Sogenannte Kompilierungsfehler. Z. B. Syntaxfehler wie fehlendes Semikolon. 🤔

Brauchen Sie weder lesen noch schreiben können. 😄

Was Sie in Java tun werden

Sie schreiben eine Quellcode-Datei, kompilieren sie mit dem `javac`-Compiler und führen dann den kompilierten Bytecode in einer virtuellen Java-Maschine aus.

Zum Beispiel im OneCompiler und später in einem Texteditor. 🧑💻

```
import java.awt.*;
import java.awt.event.*;

class Party {
    public void buildInvite() {
        Frame f = new Frame();
        Label l = new Label("Party bei Tim");
        Button b = new Button("Sicher doch");
        Button c = new Button("Ohne mich");
        Panel p = new Panel();
        p.add(l);
    } // mehr Code ....
}
```

Quelle

1

Geben Sie Ihren Quellcode ein.

Speichern Sie die Datei als **Party.java**.

```
Datei Bearbeiten Fenster Hilfe Fluchen
%javac Party.java
```

Compiler

2

Kompilieren Sie die Datei **Party.java**, indem Sie `javac` (die Compiler-Anwendung) ausführen. Wenn Sie keine Fehler gemacht haben, erhalten Sie ein zweites Dokument namens **Party.class**.

Die vom Compiler erzeugte Datei **Party.class** enthält Bytecode.

```
Method Party()
  0 aload_0
  1 invokespecial #1 <Method java.lang.Object()>
  4 return

Method void buildInvite()
  0 new #2 <Class java.awt.Frame>
  3 dup
  4 invokespecial #3 <Method java.awt.Frame()>
```

Ausgabe (Code)

3

Kompilierter Code: **Party.class**

```
Datei Bearbeiten Fenster Hilfe Fluchen
%java Party

Party bei Tim
Sicher doch Ohne mich
```

Virtuelle Maschinen

4

Führen Sie das Programm aus, indem Sie die Java Virtual Machine (JVM) mit der Datei **Party.class** starten. Die JVM übersetzt den Bytecode in etwas, das die zugrunde liegende Plattform versteht, und führt Ihr Programm aus.

Am Anfang im OneCompiler. 😓

! ? Quiz

🔧 Wie Java funktioniert

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Beim Programmieren geben Sie dem Computer sozusagen Befehle, was er tun soll. Deshalb spricht man auch von imperativer Programmierung.

Richtig!

Neben Java-Quellcode müssen Sie auch in der Lage sein, Java-Bytecode zu schreiben.

Der Java-Compiler übersetzt für Sie den Quellcode in den entsprechenden Bytecode.

Der Java-Bytecode wird von der Java Virtual Machine auf verschiedenen Geräten wie Computern und Mobiltelefonen ausgeführt.

Richtig!

Sie schreiben Ihre Party-Klasse in die Datei `Party.class`. Der Compiler übersetzt den darin enthaltenen Quellcode und erzeugt die Datei `Party.java`. Sie können das Programm ausführen, indem Sie die JVM mit dieser Datei starten.

`Party.java` ↔ `Party.class`

Quellcode schreiben Sie immer in einem Texteditor. Dies ist die schnellste und komfortabelste Methode.

Für den Anfang ist OneCompiler sehr hilfreich. Für Fortgeschrittene ist eine IDE zu empfehlen.

**Ihr – vielleicht erster –
Java-Code**



Wie Java funktioniert

- Versuchen Sie zu zweit oder zu dritt herauszufinden, was die einzelnen Codezeilen tun!
- Vielleicht wissen Sie es schon.
- Vielleicht können Sie es erraten.
- Vielleicht können Sie es auch ableiten.

Tauchen Sie ein: eine Kostprobe

Spitzen Sie Ihren Bleistift

Sehen Sie, wie einfach es ist, Java zu schreiben

Antworten auf Seite 6.

Versuchen Sie zu erraten, was die einzelnen Codezeilen tun (die Antworten finden Sie auf der folgenden Seite).

```
int size = 27;

String name = "Fido";

Dog myDog = new Dog(name, size);

x = size - 5;

if (x < 15) myDog.bark(8);

while (x > 3) {
    myDog.play();
}

int[] numList = {2, 4, 6, 8};

System.out.print("Hello");

System.out.print("Dog: " + name);

String num = "8";

int z = Integer.parseInt(num);

try {
    readTheFile("myFile.txt");
}

catch (FileNotFoundException ex) {
    System.out.print("File not found.");
}
```

Deklariert eine Integer-Variablen namens `size` und weist ihr den Wert 27 zu.

Ist x (mit dem Wert 22) kleiner als 15, wird der Hund angewiesen, achtmal zu bellen.

Gibt »Hello« aus ... wahrscheinlich auf der Kommandozeile.

F: Die Namenskonventionen für die verschiedenen Java-Versionen sind verwirrend. Es gab JDK 1.0, 1.2, 1.3 und 1.4. Dann kam ein Sprung zu J2SE 5.0, dann wurde zu Java 6 und Java 7 gewechselt. Als ich das letzte Mal nachgesehen habe, war Java 18 die aktuelle Version. Was ist da los?

weil neue Versionen rückwärtskompatibel sind bis zurück zu 1.0. Allerdings verwirrt eine Versionsnummer, die anders ist als die normalerweise genutzte. Daher ist seit Java 9 die offizielle Versionsnummer die Zahl ohne die vorangestellte 1. Java 9 ist also tatsächlich Version 9 und nicht Version 1.9.

A: Die Versionsnummern haben sich in den letzten 25 Jahren stark verändert! Die Buchstaben (J2SE/SE) können wir getrost ignorieren, da sie inzwischen kaum noch benutzt werden. Bei den Zahlen ist es etwas komplizierter.

Wir verwenden hier für die Versionen 1.0 bis 1.4 die übliche Konvention. Ab Version 5 lassen wir die vorangestellte 1 weg.

Technisch gesehen, war Java 5.0 eigentlich Java 1.5 – und ebenso Java 6 = 1.6, 7 = 1.7 und 8 = 1.8. Theoretisch hat Java immer noch die Version 1.x,

Außerdem gab es seit der Veröffentlichung von Version 9 alle sechs Monate ein neues Java-Release mit einer eigenen »Haupt«-Versionsnummer, was erklärt, warum wir in so kurzer Zeit von Version 9 zu Version 19 gelangt sind.

Sehen Sie, wie einfach es ist, Java zu schreiben

```
int size = 27;
String name = "Fido";
Dog myDog = new Dog(name, size);
x = size - 5;
if (x < 15) myDog.bark(8);

while (x > 3) {
    myDog.play();
}

int[] numList = {2, 4, 6, 8};
System.out.print("Hello");
System.out.print("Dog: " + name);
String num = "8";
int z = Integer.parseInt(num);

try {
    readTheFile("myFile.txt");
}
catch (FileNotFoundException ex) {
    System.out.print("File not found.");
}
```



Spitzen Sie Ihren Bleistift



Antworten auf Seite 6.

Versuchen Sie zu erraten, was die einzelnen Codezeilen tun (die Antworten finden Sie auf der folgenden Seite).

Deklariert eine Integer-Variable namens »size« und weist ihr den Wert 27 zu.
Ist x (mit dem Wert 22) kleiner als 15, wird der Hund angewiesen, achtmal zu bellen.
Gibt »Hello« aus ... wahrscheinlich auf der Kommandozeile.

Hurra,
ich darf
mich
irren!



Sehen Sie, wie einfach es ist, Java zu schreiben

```
int size = 27;
String name = "Fido";
Dog myDog = new Dog(name, size);
x = size - 5;
if (x < 15) myDog.bark(8);

while (x > 3) {
    myDog.play();
}
```

```
int[] numList = {2, 4, 6, 8};
System.out.print("Hello");
System.out.print("Dog: " + name);
```

Keine Sorge, wenn Sie das hier jetzt noch nicht verstehen!
Alles, was hier steht, wird in diesem Buch ausführlich erklärt (das meiste auf den ersten 40 Seiten). Wenn Java einer Sprache ähnelt, mit der Sie schon gearbeitet haben, wird Ihnen einiges recht leichtfallen. Falls nicht, machen Sie sich keine Sorgen. Wir schaffen das schon ...

Deklariert eine Integer-Variable namens >>size<< und weist ihr den Wert 27 zu.
Ist x (mit dem Wert 22) kleiner als 15, wird der Hund (Dog) angewiesen, achtmal zu bellen.
Deklariert eine Liste von Integer-Variablen namens >>numList<< und packt 2, 4, 6 und 8 in die Liste.
Gibt >>Hello<< aus ... wahrscheinlich auf der Kommandozeile.
Gibt auf der Kommandozeile >>Dog: Fido<< aus (das ist von >>name<< ist Fido)


```
while (x > 3) {  
    myDog.play();  
}
```

Die Schleife wird so lange durchlaufen, wie x größer als 3 ist ...

Sagt dem Hund (myDog), dass er spielen soll (was auch immer DAS für einen Hund bedeutet).

Das sieht nach dem Ende der Schleife aus – alles in { } wird in der Schleife ausgeführt.

```
int[] numList = {2, 4, 6, 8};
```

```
System.out.print("Hello");
```

```
System.out.print("Dog: " + name);
```

```
String num = "8";
```

```
int z = Integer.parseInt(num);
```

Gibt >>Hello<< aus ... wahrscheinlich auf der Kommandozeile.

```
try {
```

```
    readTheFile("myFile.txt");
```

```
}
```

```
catch (FileNotFoundException ex) {
```

```
    System.out.print("File not found.");
```

```
}
```



Codestruktur

Verschachtelung

🧱 Codestruktur

- Sie packen Quellcode in eine **Quelldatei**.
- Sie packen eine **Klasse** in eine Quelldatei.
- Sie packen **Methoden** in eine Klasse.
- Sie packen **Anweisungen** („Befehle“) in eine Methode.

Spoiler-Alarm:
Anweisungen können
verschachtelt sein.

Quelldatei



Klassen



Codestruktur

- Quelldateien („.java-Dateien“) enthalten in der Regel eine **Klassendefinition**.
- Klassen sind die wesentlichen Bausteine von Java-Programmen.
- Eine einzelne Klasse ist die Ausnahme, zum Beispiel bei sehr kleinen Programmen.
- Die Klasse umschließt ihren Klassenkörper in geschweiften Klammern: { ... }

```
public class Dog {
```

1

Klasse

Wie heißt
diese
Klasse?



Methoden

Codestruktur

- Eine Klasse enthält gewöhnlich eine oder mehrere **Methoden**.
- Die Methode bark der Klasse Dog enthält Anweisungen, wie ein Hund bellt.
- Methoden stehen innerhalb der geschweiften Klammern der Klasse.

```
public class Dog {  
    void bark() {  
  
    }  
}
```

Methode

Welche Methoden
könnte es noch
geben? 



Anweisungen

Codestruktur

- Die **Anweisungen** der Methode stehen wiederum im Inneren der Methode, also innerhalb ihrer geschweiften Klammer.
- Diese Anweisungen geben an, was passiert, wenn die Methode ausgeführt wird.
- Eine Methode ist sozusagen eine Art Unterprogramm in der Klasse.

```
public class Dog {  
    void bark() {  
        statement1;  
        statement2;  
    }  
}
```

Anweisungen



! ? Quiz

📦 Codestruktur

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Eine Klasse enthält Methoden, die wiederum Anweisungen enthalten. Das Ganze packen Sie in eine Quelldatei mit der Endung `.java`.

Richtig!

Ein Java-Programm besteht immer aus
genau einer Klasse.

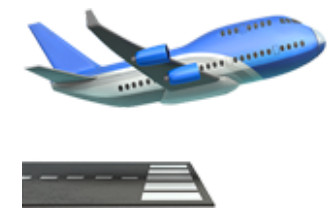
Das ist eher die Ausnahme. Größere Programme bestehen aus mehreren Klassen als Grundbausteine.

Die Methoden einer Klasse stehen innerhalb der geschweiften Klammern dieser Klasse.

Richtig!

Eine Methode ist eine Art Oberprogramm einer Klasse.

Eine Klasse enthält Methoden, sie sind also sozusagen Unterprogramme der Klasse.



main()-Methode

Startpunkt

main()-Methode

- Die Java Virtual Machine (JVM) sucht nach einer Methode namens **main**, um Ihr Programm zu starten.
- Sie teilen der JVM als Argument mit, in welcher Klasse diese Methode zu finden ist.
- Die **main()**-Methode muss die folgende Signatur haben:

```
public static void main(String[] args)
```



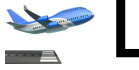
Startpunkt

main()-Methode

- Die JVM führt alle Anweisungen zwischen den geschweiften Klammern aus:

```
public static void main(String[] args) {  
    // Hier stehen Ihre Anweisungen...  
}
```

- Die JVM führt das Programm solange aus, bis alle Anweisungen ausgeführt wurden.

-  Die `main()`-Methode ist der Ort, an dem die Ausführung Ihres Programms beginnt.





```
1 public class MyFirstApp {  
2  
3     public static void main(String[] args) {  
4         System.out.println("I Rule!");  
5         System.out.println("The World");  
6     }  
7  
8 }  
9
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

The Java Playground

RunClearSamples ▾

Java 25
+ Preview Features

About ▾

```
1 System.out.println("I Rule!");  
2 System.out.println("The World")
```

Console > ☐ Detailed output

! ? Quiz

✈️ **main()-Methode**

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Die Java Virtual Machine (JVM) sucht nach einer Methode namens `start`, um Ihr Programm zu starten.

Diese Methode muss `main` heißen.

Die `main()`-Methode ist der Ort, an dem die Ausführung Ihres Programms beginnt.

Richtig!

Im Java Playground benötigen Sie keine `main()`-Methode, um Ihren Code auszuführen.

Richtig!

Im OneCompiler passen Sie die vordefinierte `main()`-Methode so an, dass sie Ihren Code enthält.

Richtig!

 **Zielgerade**

↔ Teach Back

📖 Einstieg in Java

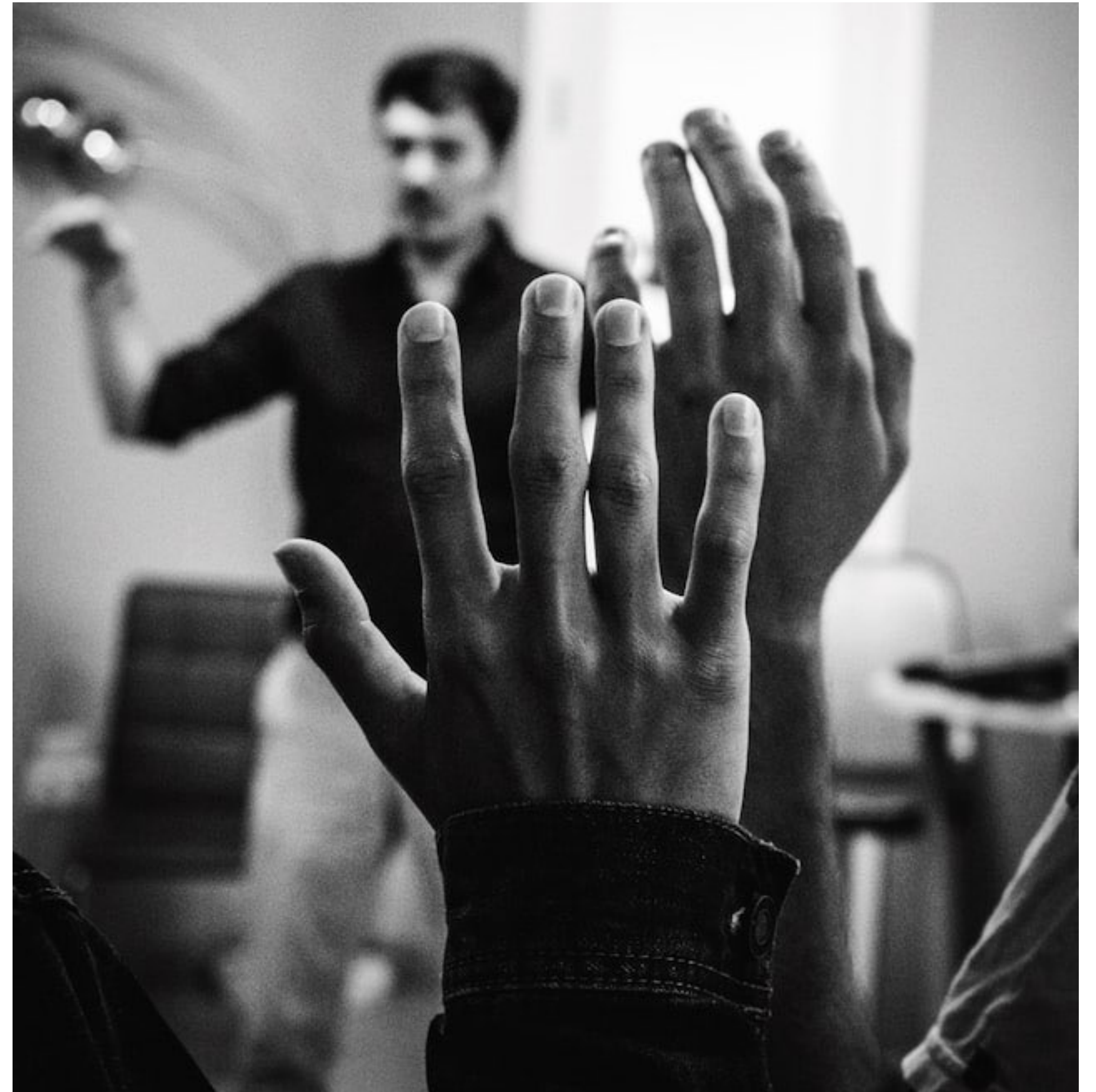
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Einstieg in Java**?
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Einstieg in Java



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

